

BỘ MÔN KHOA HỌC DỮ LIỆU

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

LAB 08 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT



HỌC KỲ 1 – NĂM HỌC 2024-2025



KHÓA K28 & K29

#Thực hành 01:

#Thuật toán thông minh bầy đàn

- +Thuật toán trí thông minh bầy đàn mô phỏng như sau: Trí thông minh bầy đàn bắt nguồn từ hành vi phân tán của các sinh vật khác nhau đang tồn tại;
- +Các hệ thống có tổ chức ảnh hưởng đến việc phân cấp theo bầy đàn bao gồm: đàn chim, đàn cá và đàn côn trùng .v.v..
- +Thuật toán này có thể áp dụng theo quy tắc đơn giản, được chia sẻ bởi tất cả các thực thể và cho phép giải quyết các vấn đề tối ưu hóa dựa trên sự hợp tác lẫn nhau, sử dụng sự tương tác giữa các cá nhân, gọi là tác nhân.
- +Thuật toán thông minh về đàn kiến được sử dụng phổ biến nhất, so sánh ảnh hưởng giữa thuật toán tối ưu hóa bầy đàn (PSO) với tối ưu hóa đàn kiến (ACO).

#Thực hành 01:

#Tối ưu hóa bầy đàn (PSO)

- +Tối ưu hóa bầy đàn là một kỹ thuật tối ưu hóa, trong đó một tập hợp các giải pháp tiềm năng sử dụng hành vi xã hội của tính chất bầy đàn chim hoặc đàn cá để giải quyết vấn đề nêu trên.
- +Bên trong bầy đàn, mỗi phân đoạn được coi là một hạt có tiềm năng đưa ra giải pháp.
- +Các hạt trong không gian tìm kiếm theo bầy đàn và thay đổi vị trí của chúng trên các bước đó bằng kiến thức của chính chúng, như kiến thức của tất cả các hạt khác ở gần đó.

#Thực hành 01:

#thuật toán PSO – thuật toán bầy đàn;

Thuật toán Tối ưu hóa bầy đàn (PSO) gồm các bước sau:

(1) Khởi tạo: Xác định miền và khởi tạo ngẫu nhiên quần thể các đàn (giải pháp ứng viên) trong miền đó.

(2) Đánh giá: Đánh giá độ phù hợp của từng bầy đàn (dựa trên vị trí của đàn).

(3) Cập nhật điểm tốt nhất cá nhân của đàn: Cập nhật vị trí tốt nhất cá nhân của từng đàn dựa trên giá trị độ phù hợp của hạt đó.

(4) Cập nhật điểm tốt nhất toàn cục: Cập nhật vị trí tốt nhất toàn cục dựa trên vị trí tốt nhất của tất cả các đàn.

(5) Cập nhật vận tốc và vị trí: Cập nhật vận tốc và vị trí của từng đàn dựa trên vị trí và vận tốc hiện tại của đàn đó, cũng như vị trí tốt nhất cá nhân và toàn cục.

(6) Quay lên lặp Bước 2 số lần lặp cố định cho đến khi đàn được tiêu chí dừng. Kết thúc.

(ví dụ: đạt được số lần lặp tối đa, đạt được giá trị phù hợp mục tiêu, v.v.).

#Thực hành 01:

```
import numpy as np
def rastrigin(x):
    return 10 * len(x) + sum([(xi ** 2 - 10 * np.cos(2 * np.pi * xi)) for
xi in x])
class Particle:

    def __init__(self, bounds):
        self.position = np.random.uniform(bounds[:, 0], bounds[:, 1],
len(bounds))
        self.velocity = np.random.uniform(-1, 1, len(bounds))
        self.pbest_position = self.position.copy()
        self.pbest_value = float('inf')

    def update_velocity(self, gbest_position, w=0.5, c1=1.0, c2=1.5):
        r1 = np.random.rand(len(self.position))
        r2 = np.random.rand(len(self.position))
        cognitive_velocity = c1 * r1 * (self.pbest_position -
self.position)
        social_velocity = c2 * r2 * (gbest_position - self.position)
        self.velocity = w * self.velocity + cognitive_velocity +
social_velocity

    def update_position(self, bounds):
        self.position += self.velocity
        self.position = np.clip(self.position, bounds[:, 0], bounds[:, 1])
```

#Thực hành 01:

```
def particle_swarm_optimization(objective_func, bounds, n_particles=30,
max_iter=100):
    particles = [Particle(bounds) for _ in range(n_particles)]
    gbest_position = np.random.uniform(bounds[:, 0], bounds[:, 1],
len(bounds))
    gbest_value = float('inf')
    for _ in range(max_iter):
        for particle in particles:
            fitness = objective_func(particle.position)
            if fitness < particle.pbest_value:
                particle.pbest_value = fitness
                particle.pbest_position = particle.position.copy()

            if fitness < gbest_value:
                gbest_value = fitness
                gbest_position = particle.position.copy()

        for particle in particles:
            particle.update_velocity(gbest_position)
            particle.update_position(bounds)
    return gbest_position, gbest_value
bounds = np.array([[-5.12, 5.12]] * 10)
#PSO - tối ưu hoá bầy đàn
best_solution, best_fitness = particle_swarm_optimization(rastrigin,
bounds, n_particles=30, max_iter=100)
#xuất ra giải pháp & thể lực tốt nhất
print("Best solution:", best_solution)
print("Best fitness:", best_fitness)
```

#Thực hành 01:

#out:

```
Best solution: [-2.98485865e+00  5.56005126e-06 -9.94963108e-01  9.94927136e-01  
-2.96206091e-06  1.98991071e+00  9.94954809e-01 -4.36667850e-06  
-9.94952119e-01 -9.94956162e-01]  
Best fitness: 17.90922794462658
```

#nhận xét: input, output?

#Thực hành 02:

#Tối ưu hóa đàn kiến (ACO)

- +Tối ưu hóa đàn kiến được lấy ý tưởng từ hành vi tìm kiếm thức ăn của loài kiến.
- + Đàn kiến tìm đường đi ngắn nhất giữa tổ và nguồn thức ăn bằng cách tiết ra pheromone để hướng dẫn những con kiến khác đi theo đường đi đó.
- +Cách triển khai cơ bản của ACO cho Bài toán đường đi nhân viên bán hàng (TSP)

#ACO - tối ưu hóa đàn kiến

```
import numpy as np

class Ant:
    def __init__(self, n_cities):
        self.path = []
        self.visited = [False] * n_cities
        self.distance = 0.0
    def visit_city(self, city, distance_matrix):
        if len(self.path) > 0:
            self.distance += distance_matrix[self.path[-1]][city]
        self.path.append(city)
        self.visited[city] = True
    def path_length(self, distance_matrix):
        return self.distance + distance_matrix[self.path[-1]][self.path[0]]
```


#Thực hành 02:

```
def ant_colony_optimization(distance_matrix, n_ants=10, n_iterations=100,
alpha=1, beta=5, rho=0.1, Q=10):
    n_cities = len(distance_matrix)
    pheromone = np.ones((n_cities, n_cities)) / n_cities
    best_path = None
    best_length = float('inf')
    for _ in range(n_iterations):
        ants = [Ant(n_cities) for _ in range(n_ants)]
        for ant in ants:
            ant.visit_city(np.random.randint(n_cities), distance_matrix)
            for _ in range(n_cities - 1):
                current_city = ant.path[-1]
                probabilities = []
                for next_city in range(n_cities):
                    if not ant.visited[next_city]:
                        pheromone_level =
pheromone[current_city][next_city] ** alpha
                        heuristic_value = (1.0 /
distance_matrix[current_city][next_city]) ** beta
                        probabilities.append(pheromone_level *
heuristic_value)
                else:
                    probabilities.append(0)
                probabilities = np.array(probabilities)
                probabilities /= probabilities.sum()
                next_city = np.random.choice(range(n_cities),
p=probabilities)
            ant.visit_city(next_city, distance_matrix)
```

#Thực hành 02:

```
for ant in ants:
    length = ant.path_length(distance_matrix)
    if length < best_length:
        best_length = length
        best_path = ant.path

pheromone *= (1 - rho)
for ant in ants:
    contribution = Q / ant.path_length(distance_matrix)
    for i in range(n_cities):
        pheromone[ant.path[i]][ant.path[(i + 1) % n_cities]] +=
contribution
    return best_path, best_length
#ma trận khoảng cách giữa 5 thành phố;
distance_matrix = np.array([
    [0, 2, 2, 5, 7],
    [2, 0, 4, 8, 2],
    [2, 4, 0, 1, 3],
    [5, 8, 1, 0, 6],
    [7, 2, 3, 6, 0]
])

#ACO- tối ưu hoá đàn kiến
best_path, best_length = ant_colony_optimization(distance_matrix)
#xuất ra đường đi và độ dài đường đi tối ưu
print("Best path:", best_path)
print("Best length:", best_length)
```

#Thực hành 02:

#out:

Best path: [1, 4, 2, 3, 0]

Best length: 13.0

#nhận xét: +input, output?

+Vẽ đồ thị và biểu diễn đường đi qua

5 thành phố - tìm đường đi ngắn nhất?

#Thực hành 03:

#Triển khai thuật toán REINFORCE

Giả sử tình huống đơn giản trong đó đặc điểm trường hợp một số kỹ năng chơi trò chơi nhất định. Mạng lưới thần kinh tạo ra xác suất thực hiện một số hành động nhất định có thể đóng vai trò là chính sách. Bằng cách chơi trò chơi, tác nhân thu thập quỹ đạo, tính toán lợi nhuận, sử dụng độ dốc chính sách để thay đổi các tham số của mạng thần kinh và sau đó lặp lại quy trình.

#Thực hành 03:

#Thuật toán

B.1: Thiết lập các tham số chính sách như thiết lập chính sách ban đầu. Biểu diễn tham số hoặc mạng lưới thần kinh có thể được sử dụng cho việc này.

B.2: Đường dẫn thu thập: như để thu thập một tập hợp các quỹ đạo-chuỗi các trạng thái, hành động và phần thưởng-hãy thực thi chính sách hiện tại trong môi trường.

B.3: Xác định lợi nhuận: như tính toán lợi nhuận hoặc tổng số phần thưởng được chiết khấu từ mỗi trạng thái trở đi cho từng trạng thái trong quỹ đạo.

B.4: Tính toán độ dốc chính sách: như xác định độ dốc của lợi nhuận dự kiến đối với các tham số của chính sách. Để làm điều này, phải tính toán độ dốc của nhật ký khả năng của quá trình hành động đã chọn.

B.5: Làm mới các tham số chính sách: như các tham số chính sách phải được cập nhật theo cách khiến các hành động có nhiều khả năng mang lại lợi nhuận cao hơn. Độ dốc đi lên thường được sử dụng cho việc này.

B.6: Lặp lại: Các bước từ B.2 đến B.5 và được lặp lại nhiều lần.

#Thực hành 03:

```
import numpy as np
#bước khởi tạo
def initialize_policy(num_actions):
    return np.random.rand(num_actions)
#các hàm functions
def collect_trajectories():
    #giả sử một quỹ đạo cố định cho mỗi tập
    states = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    actions = [0, 1, 2]
    rewards = [0.1, 0.5, 0.2]
    return states, actions, rewards
def compute_returns(rewards):
    #hàm tính toán lợi nhuận từ phần thưởng
    pass
def compute_policy_gradient(states, actions, returns):
    #hàm tính toán độ dốc chính sách
    pass
def update_policy_parameters(policy_gradient):
    #hàm cập nhật các tham số chính sách bằng cách sử dụng gradient
    pass
```

#Thực hành 03:

```
num_actions = 3
policy_parameters = initialize_policy(num_actions)
#số vòng lặp training -huấn luyện
num_episodes = 1000
for episode in range(num_episodes):
    #thu thập quỹ đạo
    states, actions, rewards = collect_trajectories()
    #tính toán lợi nhuận
    returns = compute_returns(rewards)
    #tính toán độ dốc chính sách
    policy_gradient = compute_policy_gradient(states, actions, returns)
    #cập nhật các tham số chính sách
    update_policy_parameters(policy_gradient)

#chính sách cuối cùng
final_policy_parameters = policy_parameters

print("Final Policy Parameters:", final_policy_parameters)
#out:
```

Final Policy Parameters: [0.91411562 0.32903155 0.96290135]



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

